**CST1500 – Laboratory Worksheet - Week 12**

These weekly laboratory worksheets provide information and exercises to complete during class or at your own leisure. The practical exercises will assist you with your coursework as well as provide you with valuable knowledge and skills. Please do your best to keep up with the weekly laboratory worksheets in order to maximise your potential for good marks and satisfactory progress in the module.

Exercises are denoted in red writing. For example:

**EX 1: Identification of PC parts (identify as many as you can).**

## Contents

Lecture videos to watch this week ..... 1

Introduction to Arduino ..... 1

Arduino Basic Examples (The Blinking Led) ..... 11

## Lecture videos to watch this week

Lecture 22 Revision

## Introduction to Arduino

**Please use:**

https://www.tinkercad.com/

**Lab Objectives**

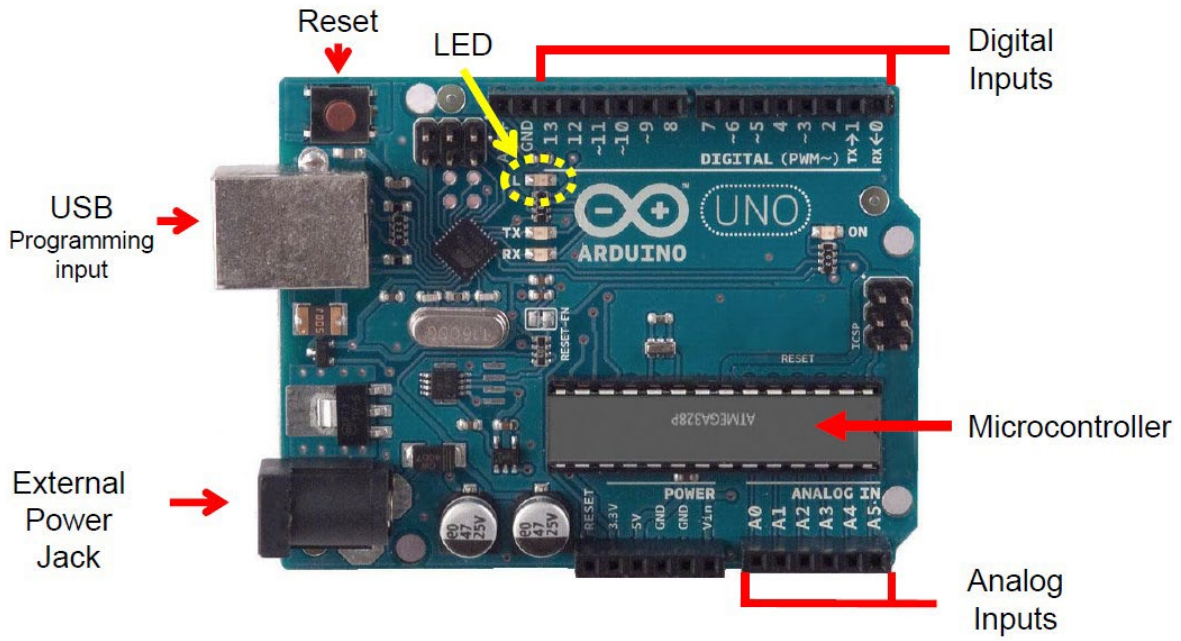
This lab is an introduction to programming with the Arduino and its basic functions. It also teaches you how to control digital outputs based on analog/digital input readings.

**Background**

Any electronic product that needs computation or interfacing with other computers first requires a quick prototyping of the concept using simple tools. Arduino is an open source hardware prototyping platform designed around a popular microcontroller family, and it includes a simple software development environment. Besides prototyping, you can also use Arduino for the development of your own do-it-yourself (DIY) projects. Arduino bridges the computational world with the physical world by letting you simply connect the sensors and actuators with a computer. Basically, you can write code to monitor and control various electronic components in your daily life by using Arduino's input/output pins and microcontroller. Examples of these components include motors, thermostats, lights, switches, and many more.

## What is Arduino?

Arduino is one of the most popular open-source, single-board microcontrollers. But what does that really mean? Well, it means that it consists of a single board that provides all the circuitry you need to control electronic devices that you hook up to the board. And it means that information about the hardware and software that controls it is readily available. With it, you can build and connect electronic devices and control them with code that you write. Sweet! As you work with your Arduino, remember that Arduino has a huge user community, so there are lots of resources available.
Below is a labelled diagram of an Arduino Uno SMD microcontroller. Use it to familiarise yourself with the basic components of the Arduino, and return to it throughout the assignment as needed. You don't need to memorize every component, but it is useful to have a general sense of what's going on with the microcontroller.




## EX 1: Arduino Board identification

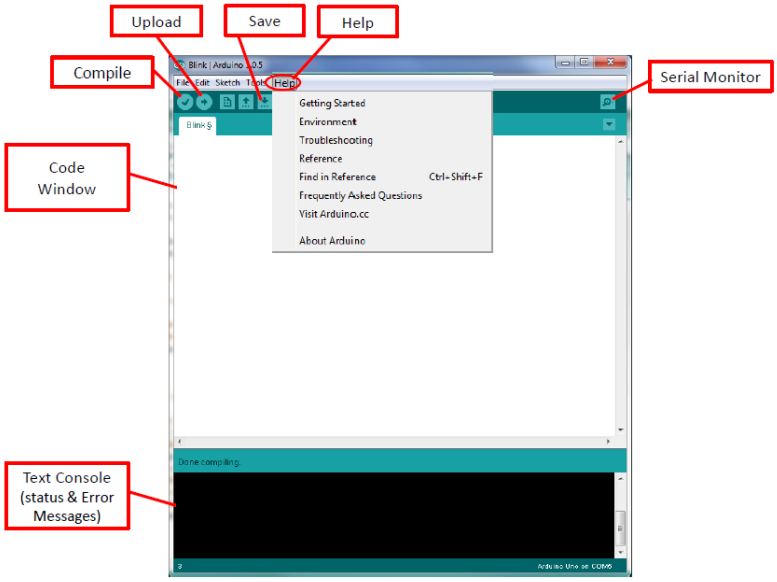
Based on your knowledge and online research, identify Arduino board components and discuss their roles/tasks to your lab tutor, SLA or classmate (or make some notes).

## Installation

One reason the Arduino system was chosen for this course is because of its low cost and it can demonstrate many things through its application. The software is free and development boards are available for under £25. If you would like to install the Arduino software on your computer, first simply go to http://arduino.cc/en/Main/Software and download the current version of the software for your operating system. It is available for Windows, Mac and Linux.

## IDE - Integrated Development Environment

Once the board is installed, it's time to open the Arduino IDE. It is fairly simple when compared to larger desktop C language development systems. Explore the Arduino IDE and check all available features and examples.



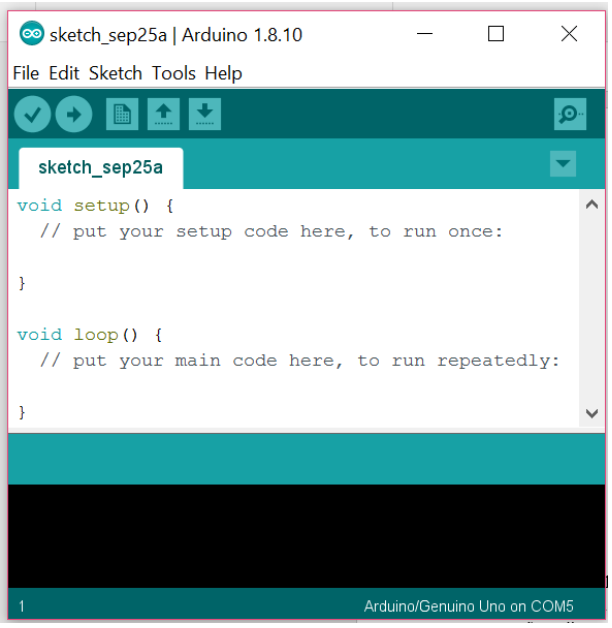

## Basic structure of a sketch

• Setup (absolutely necessary for the program)

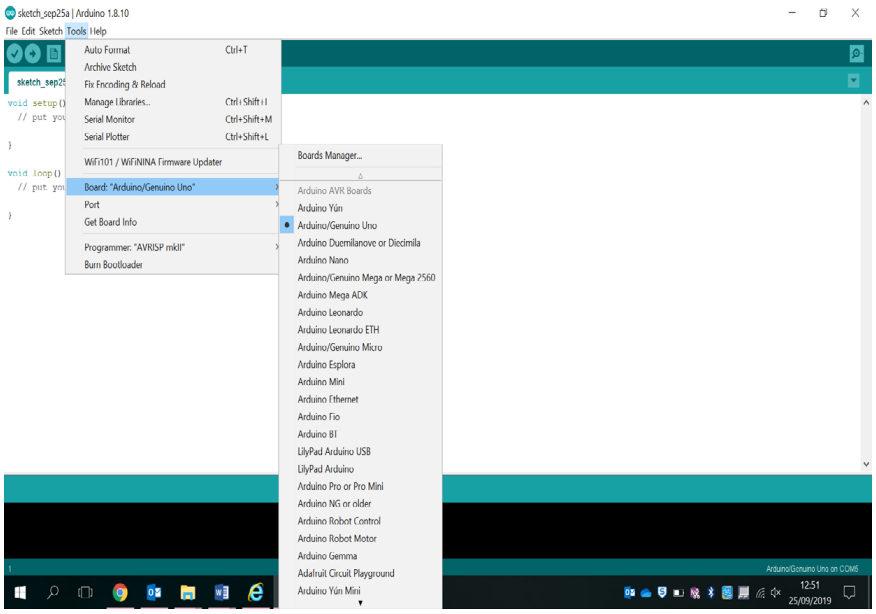
The setup will be performed only once. Here you are telling the program for example what Pin (slot for cables) should be an input and what should be an output on the boards. Defined as Output: The pin should put out a voltage. For example: With this pin an LED is meant to light up. Defined as an Input: The board should read out a voltage. For example: A switch is actuated. The board recognised this, because it gets a voltage on the Input pin.

• Loop (absolutely necessary for the program)

This loop part will be continuously repeated by the board. It assimilates the sketch from beginning to end and starts again from the beginning and so on.




On the other hand, The two most important items here are Board (from which you select your particular board) and Serial Port (from which you select the COM port to which your board is connected). Failure to set these correctly will result in unsuccessful programming of your board.




## Coding

Ordinarily, all code will be contained within a single file. When you create a new project, or “sketch” in Arduino, a new directory will be created for it. Unlike ordinary C language source files, Arduino sketches utilize a “.ino” extension instead of “.c”. Further, the normal main() entry point is not used. In essence, the Arduino system has already written main() for you and it looks something like this:

```
void main()
{
     init();
     setup();
     while(1)
          loop();
}
```

The first function call is where various system initialisations occur such as allocating and pre-setting timers, the ADC system, etc. This has already been written for you. The other two calls, setup() and loop(), are for you to write. setup() is your own initialisation function (i.e., things you need to do just once at start-up) and loop() is the portion that will be called repeatedly.

One of the issues with embedded programming is that you don't have a console for input and output via scanf() and printf(). Normally, an embedded program won't need these but they are very useful when it comes to debugging a program. Without printf(), how can you insert test points in your code to check progress, values, etc? One technique is to simply flash an LED at specific points in the program. The Arduino boards include a small LED on one of the ports for this purpose. While this is useful, it is limited, so a second and more powerful technique involves two-way communication with the host computer. On the Arduino, this is done via the Serial library. The IDE includes a serial monitor under the tools menu. Selecting this will open a small window. You can use it to send data to the board or send data from the board back to the host computer.

It is important to be mentioned that there are two common languages used with Arduino (c code and python). There isn't huge difference between the two programming languages, so you may use your preference. Below are some common comparisons:

**Arduino - C vs Python**

•Static types
```
int age=18; vs age=18
```

•Statements end at semicolon, not line
```
int age=18;
int age=18
```

•Bracketed blocks
```
int main() { /* this is a block */ };

main():
     # this is a block
```

## EX 2: Compiling the Hello World example

Let's look at an example of communicating via the Serial Monitor. First, let's consider writing data from the board back to the host. Here's a "Hello World" program:

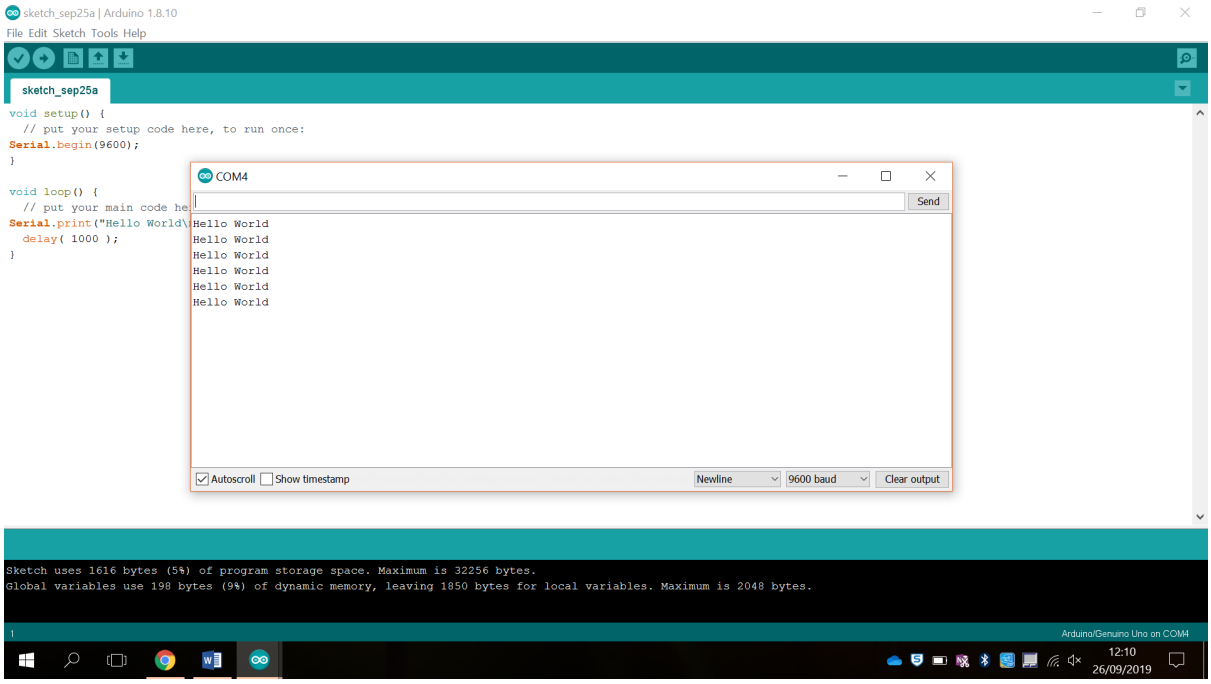
```
void setup()
{
     Serial.begin(9600);
}

void loop()
{
     Serial.print("Hello World\n");
     delay( 1000 );
}
```

In the setup function we open the serial library and set the communication speed at 9600 baud (more or less standard speed for the Arduino) using the Serial.begin() function. There is a complimentary Serial.end() function should you decide that you need the particular port pins that are used for serial communication for other purposes later on. Note that all of the calls to functions in this library will start with the library's name. The loop() functions writes our message using Serial.print() and then waits for about one second using the delay() function, the argument of which is measured in milliseconds. Once that's done, the program flow returns back to main(). Of course, main() just repeats on the loop() function so it calls it again. Our message is printed a second time, a third time and so on.

Enter the bit of code above, compile and download it to the target. Once that's done, open the Serial Monitor and look at the output. Each second you should see the message "Hello World" printed again. That's all the controller program does. Not too useful by itself here, but the power to print items back to the host will be very useful once we get going.

For simplicity to run your code you need to do the following:

1. Connect your Arduino board to your PC
2. Choose the right board and port
3. Verify/Compile the code by pressing on
4. Upload/ send your code to the board by pressing on
5. View your output via the which you may find on your right-hand side.




Get used to using the on-line documentation. Two good places to start are the Reference page at http://arduino.cc/en/Reference/HomePage and the Tutorial page at http://arduino.cc/en/Tutorial/HomePage. The Reference page includes information on all of the operators and functions built into the Arduino system. The Tutorial gives information on programming concepts and ideas. For example, here is a copy of the on-line documentation of the Serial.print() function:

`Serial.print()`

**Description**

Prints data to the serial port as human-readable ASCII text. This command can take many forms. Numbers are printed using an ASCII character for each digit. Floats are similarly printed as ASCII digits, defaulting to two decimal places. Bytes are sent as a single character. Characters and strings are sent as is. For example:

- `Serial.print(78) gives "78"`
- `Serial.print(1.23456) gives "1.23"`
- `Serial.print('N') gives "N"`
- `Serial.print("Hello world.") gives "Hello world."`

An optional second parameter specifies the base (format) to use; permitted values are BIN (binary, or base 2), OCT (octal, or base 8), DEC (decimal, or base 10), HEX (hexadecimal, or base 16). For floating point numbers, this parameter specifies the number of decimal places to use. For example:

- `Serial.print(78, BIN) gives "1001110"`
- `Serial.print(78, OCT) gives "116"`
- `Serial.print(78, DEC) gives "78"`
- `Serial.print(78, HEX) gives "4E"`
- `Serial.println(1.23456, 0) gives "1"`

- `Serial.println(1.23456, 2) gives "1.23"`
- `Serial.println(1.23456, 4) gives "1.2346"`

You can pass flash-memory based strings to Serial.print() by wrapping them with F(). For example :

- `Serial.print(F("Hello World"))`

To send a single byte, use Serial.write().

**Syntax**

```
Serial.print(val)
Serial.print(val, format)
```

**Parameters**

val: the value to print - any data type
format: specifies the number base (for integral data types) or number of decimal places (for floating point types)

**Returns**

`size_t (long): print()` returns the number of bytes written, though reading that number is optional

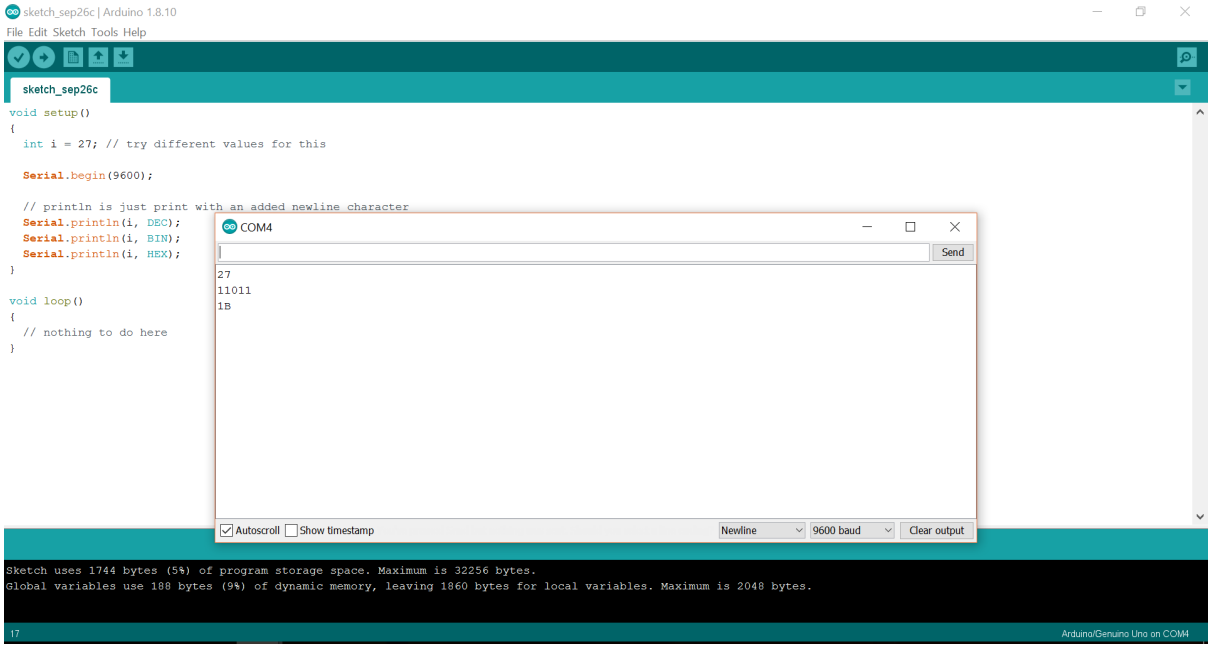
The cool thing about this function is the optional second argument that allows you to specify format information. The ability to print out values in hex and binary are particularly useful, for example, when examining bit fields.

## EX 3: Executing different data representation program

Modify your program so that it appears as follows, compile, download and run it:

```
void setup()
{
     int i = 27; // try different values for this
     Serial.begin(9600);
     // println is just print with an added newline character
     Serial.println(i, DEC);
     Serial.println(i, BIN);
     Serial.println(i, HEX);
}
void loop()
{
     // nothing to do here
}
```

## Expected output




Less common but still useful from time to time when debugging is the ability to pass values into the running program. The Serial Monitor window includes a small data entry field at the top with an associated Send button. The Serial functions of greatest interest are: `Serial.available()`, `Serial.parseFloat()` and `Serial.parseInt()`

### `Serial.available()`

### Description

Get the number of bytes (characters) available for reading from the serial port. This is data that's already arrived and stored in the serial receive buffer (which holds 64 bytes). available() inherits from the Stream utility class.

### Syntax

```
Serial.available()
```

### Parameters
none

### Returns

the number of bytes available to read

### `Serial.parseFloat()`

**Description**

`Serial.parseFloat()` returns the first valid floating point number from the Serial buffer. Characters that are not digits (or the minus sign) are skipped. parseFloat() is terminated by the first character that is not a floating point number.

`Serial.parseFloat()` inherits from the Stream utility class.

**Syntax**

`Serial.parseFloat()`

**Parameters**
none

**Returns**

Float

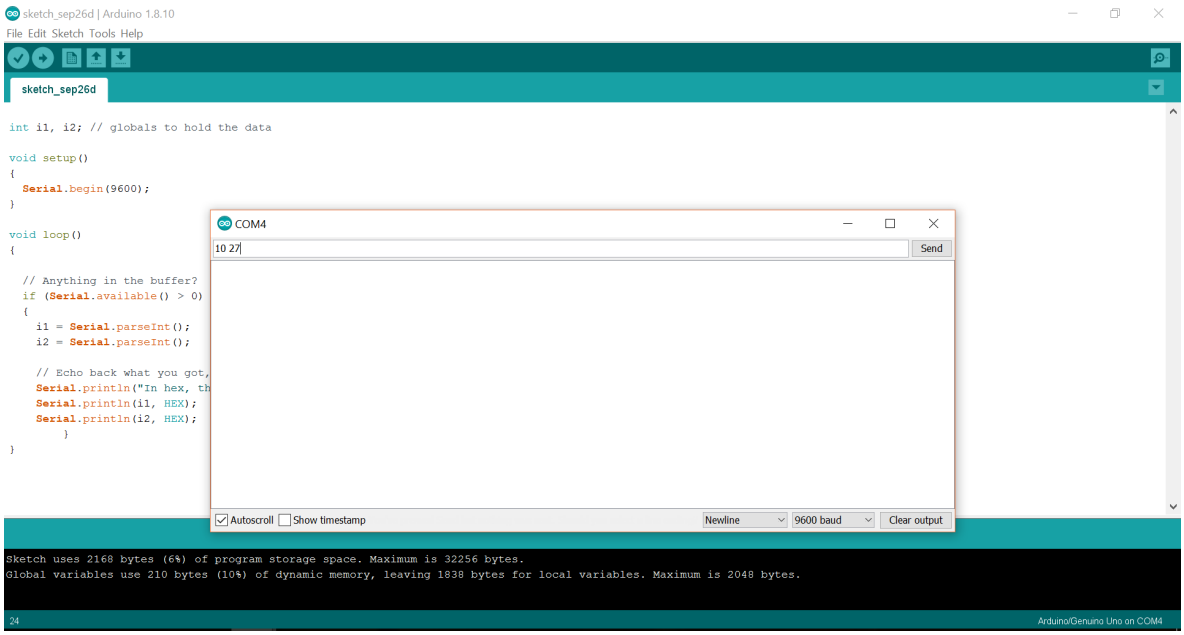
`Serial.parseInt()` is similar to `Serial.parseFloat()` but returns an integer rather than a floating point value. The two parsing functions examine the input string and begin by discarding non-numeral characters. Once a numeral is found, it continues to look at characters in the string until another non-numeral is found. It then translates the ASCII numeral bytes into a single integer value (or float, as the case may be). Note that it is possible to send multiple values at the same time.

## EX 4: Executing decimal to hex program converter

Here's an example of how they might be used to create a decimal to hex converter. Enter the following chunk of code, compile and download it.

```
int i1, i2; // globals to hold the data
void setup()
{
     Serial.begin(9600);
}
void loop()
{
     // Anything in the buffer?
     if (Serial.available() > 0)
     {
          i1 = Serial.parseInt();
          i2 = Serial.parseInt();
          // Echo back what you got, but in hex
          Serial.println("In hex, that is: ");
          Serial.println(i1, HEX);
          Serial.println(i2, HEX);
     }
}
```

Once the program has been downloaded, open the Serial Monitor and enter 10 27 into the text box at the top and then hit the Send button next to it (you could also hit the Enter key on your keyboard).




The result should be:

```
In hex, that is:
A
1B
```

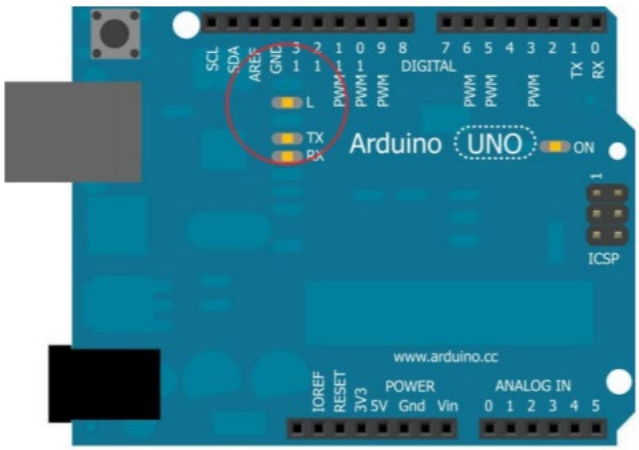
## Arduino Basic Examples (The Blinking Led)

### Lab Objectives

• Becoming more familiar with using Arduino and the IDE.
• Learning how to use the Arduino to flash an LED connected to digital pin 13.

### Getting started

On the board an LED is already built on the pin 13 (for test purposes). This LED often already blinks after connecting a new board to the computer because during manufacturing, the blink program is uploaded for test purposes. We are going to program this blinking by ourselves.




## EX 5: The Blinking Led Example

Copy/Write the code in the IDE window. For this lab session, go to File > Examples > Basics > Blink, or copy the following code into the IDE window:

```
/* Blink
Turns on an LED on for one second, then off for one second,
repeatedly. This example code is in the public domain.
*/
void setup() {
// initialize the digital pin as an output.
// Pin 13 has an LED connected on most Arduino boards:
pinMode(13, OUTPUT);
}
void loop() {
     digitalWrite(13, HIGH); // set the LED on
     delay(1000); // wait for a second
     digitalWrite(13, LOW); // set the LED off
     delay(1000); // wait for a second
}
```

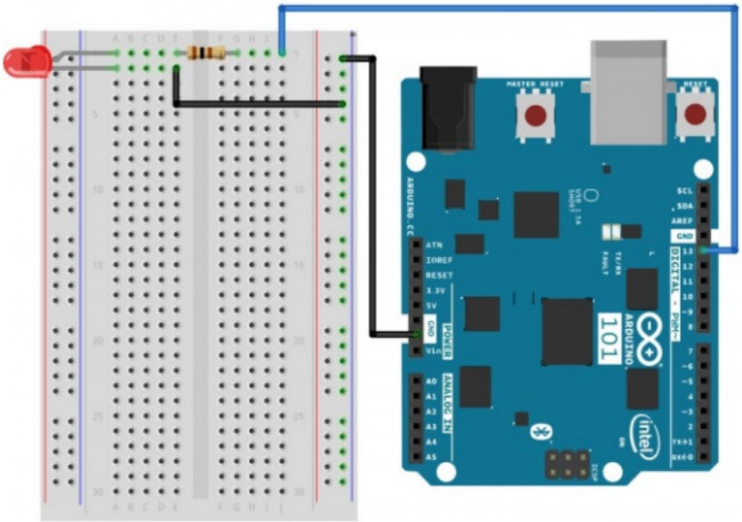
Notice that text following two slashes // or between /* */ is greyed-out and is not read by the Arduino; these are known as comments, and they serve to help the reader understand the code.

• Now go to the Tools menu of the Arduino IDE, and select the Serial Port (COM Port) to which the Arduino Board is connected to. Make sure that the Arduino is plugged into one of the USB ports.
• Go to Tools > Board. If your board says “Arduino Raheem,” make sure that (Arduino Raheem) is selected.
• Use the Upload icon of the Arduino IDE to compile and download the program to the Arduino Board.
• If your upload was successful you will get the “Done Uploading” message.
• Look at your Arduino board. You should see a yellow LED (labelled ‘L’) flashing on and off at a frequency of 0.5 Hz. This LED is wired up directly to digital pin 13. In the setup function, the pinMode function is used to specify that pin 13 is being used as an

output. Within the loop function of this code, pin 13 is written HIGH (pin 13 becomes a 5-V source) using the digitalWrite function. Next, the program waits for one second using the delay() function, which takes an argument in milliseconds. Then, pin 13 is written LOW (0V). The program again waits (does nothing) for a second. The sequence of events within loop will be repeated forever as long as the Arduino is supplied with power, while the events in setup occur only once when the program starts running.

• Now modify the code so that the flashing rate is 2 Hz (2 cycles per second).

Schematics and circuit diagram for Blinking an LED



## EX 6: Modifying the speed of the blinking led example

Now you have the option to modify the program. Example: You want the LED to blink really fast. Therefore, we will shorten the waiting time (From 1000ms to 200ms).

```
void setup() //The setup begins here
{ //A program part begins here
     pinMode(13, OUTPUT); //Pin 13 is supposed to be an output.
} //A program part is ending here.
void loop() //The main part of the program begins here
{ //program part begins here
     digitalWrite(13, HIGH); //Turn on the voltage on pin 13
     (LED on)
     delay(200); //Wait for 200 milliseconds
     digitalWrite(13, LOW); //Turn off the voltage on pin 13
     //(LED off)
     delay(200); //Wait for 200 milliseconds
} //program part ends.
```

The new modified sketch has to be uploaded to the board again. Now if everything has worked properly the LED should blink faster.

Show your result to your lab tutor, SLA or classmate.

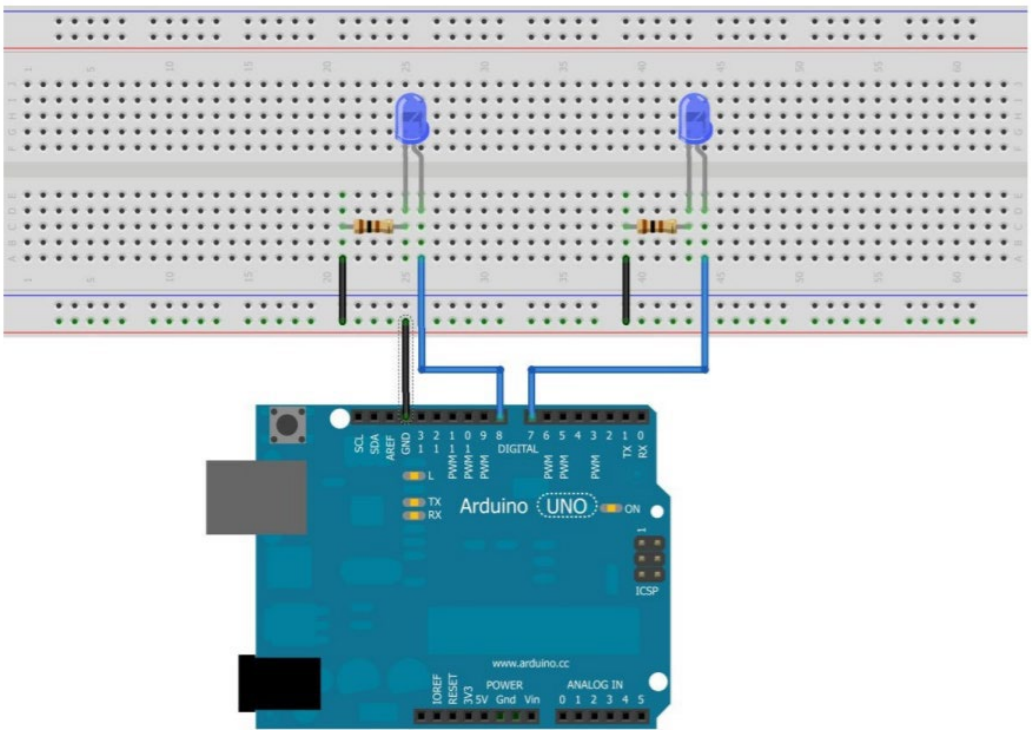
**Alternately Blinking LED Example**

## EX 7: Perform advanced programs using Arduino I

**Task**: We want to make two LEDs blink alternately.

**Required equipment**: Microcontroller / two LEDs / two resistors with 100 Ohm / Breadboard / cables

**Setup**:




**Code:**

```
void setup()
{ //We are starting with the setup
     pinMode(7, OUTPUT); //Pin 7 is defined as output
     pinMode(8, OUTPUT); //Pin 8 is defined as output
}
void loop()
{ //The main part starts
     digitalWrite(7, HIGH); //turn on the LED on pin 7
     delay(1000); //wait for 1000 milliseconds
     digitalWrite(7, LOW); //turn off the LED on pin 7
     digitalWrite(8, HIGH); //turn on the LED on pin 8
     delay(1000); //wait for 1000 milliseconds
     digitalWrite(8, LOW); //turn off the LED on pin 8
} //Here at the end of the loop the program starts
//again from the beginning of
//the loop part. So… turn on LED on pin 7..wait for
//1000 milliseconds..etc..etc..
```

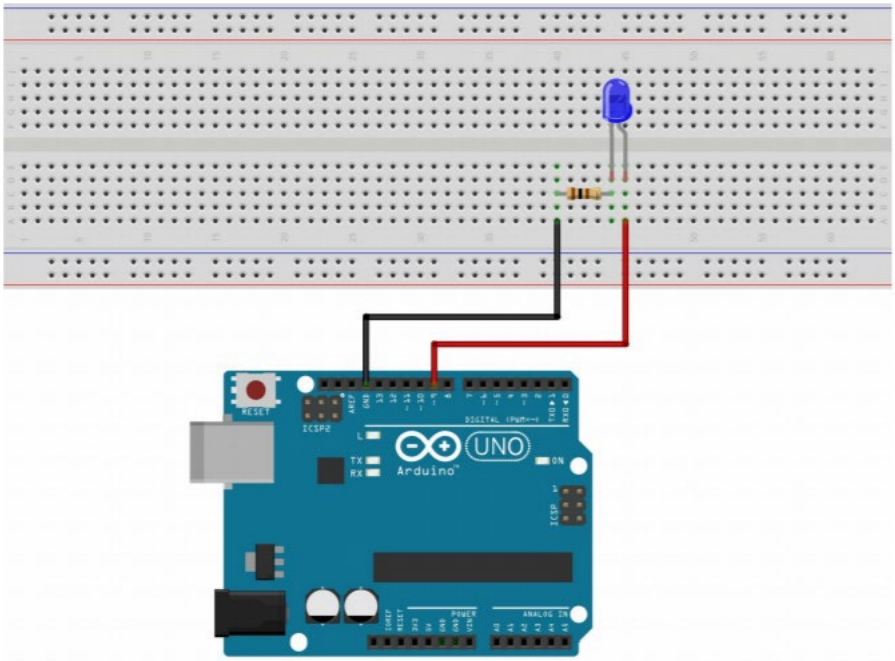
Show your result to your lab tutor, SLA or classmate.

## EX 8: Perform advanced programs using Arduino II

**Task**: A LED should pulsating get brighter and darker (fade).

**Required equipment**: Microcontroller / one LED / resistor with 100 Ohm / Breadboard / cables

**Setup**:




The Arduino is a digital microcontroller. It only knows “5 Volt on” and “5 Volt off” on its outputs. But to vary the brightness of an LED, it ought to vary the voltage on the outputs. For example, 5 Volts if the LED shines bright, 4 Volts if it shines a bit darker and so on. THIS DOESN'T WORK ON DIGITAL PINS. But there is another option. It is called pulse width modulation (PWM). PWM lets the 5 V voltage fade. The voltage is turned on and off for milliseconds. With a really high PWM the 5 V signal nearly gets constant on the pin. With a low PWM it is the other way around and the 5 V signal is barely there (this is only a reduced summary, you should look it up on the internet, if you need more information). With this PWM it is possible to get nearly the same effect on a LED as if the voltage would get varied. Not every pin on the board has the PWM function. The pins with this function are specially labelled, for example with a little wave in front of the pin number.

**Code**:

```
int LED=9; //The word “LED” stands for the number 9.
int brightness=0; //The word “brightness” stands for the value
//emitted at the PWM. The number “0” is only used as an
//initial value.
int fading=5; //”fading” sets up the speed of the fading.
void setup()
{ // The setup begins here.
pinMode(LED, OUTPUT); //The pin with the LED is supposed to be an
output.
}
void loop()
{ //The loop part begins here.
analogWrite(LED, brightness); //The function analogWrite is used to
//activate the PWM output on the pin with the
// LED. The value of the PWM is saved under
//the word “brightness”. In this case it is
// “0”.
brightness=brightness + fading;
// Now we modify the value of the PWM
//output. We add the value of the fading to
// the value of the brightness. In this
//case: //brightness = 0+5. The new value
// that is standing for brightness isn't 0
// any longer but 5. When the program has
// ran through the loop part once it will
//start over again. The next pass the value
// will be 10. After that it will be
// 15... etc.
delay(25); //The LED should only stay bright for a really short time
//like 25 milliseconds. If you reduce that time the fading will also
//get faster.
if(brightness==0 || brightness== 255){
fading= -fading;
} //The meaning of this command: If the Value of brightness will reach
//0 OR 255, the value of fading will change from negative to
//positive and the other way around. The Reason why is, that
//while the program is running through the loop part, the LED
//will shine brighter and brighter. But at the point of 255 as a
//value of the PWM, it will reach its maximum. At this point
//the LED should get darker step by step. Because of this the
//fading will be negated at this point. This means the next time
//the program runs through the loop the part
//“brightness=brightness+fading;” will make the LED get
//darker. Example: “brightness=255+(-5)”. The value of
//brightness will be 250 after that. The next time it will be
//245..etc. As the value of brightness will reach 0, the value
//of fading will get positive again.
} //with this brace, the loop part ends
```

Show your result to your lab tutor, SLA or classmate.

Feel free to further investigate the Arduino.